# Disseny i Programació Orientats a Objectes

## Memòria del projecte

# The Parking

| | |
|---|---|
| **Identificador del grup** | E-Parking-1 |
| **Membres del grup** | Judith Caralt Rico<br>Ignasi Casañas Parareda<br>Aras Mora Arnes<br>Cristina Ponsa Illas<br>Pol Martín Morales |
| **Assignatura** | Disseny i Programació Orientats a Objectes |
| **Curs** | 2025-2026 |
| **Data de lliurament** | 24/05/2026 |

# Índex


1. Resum de les especificacions del projecte ..... 2
2. Disseny de la interfície gràfica ..... 3
   2.1. Criteris generals de disseny ..... 4
   2.2. Pantalla de login ..... 5
   2.3. Pantalla de registre ..... 5
   2.4. Menú principal d'usuari normal ..... 6
   2.5. Menú principal d'administrador ..... 7
   2.6. Estat actual del parking ..... 7
   2.7. Entrada i sortida del parking ..... 8
   2.8. Gestió de reserves de l'usuari ..... 9
   2.9. Gestió de places de l'administrador ..... 10
   2.10. Gestió de reserves per part de l'administrador ..... 11
   2.11. Estadístiques ..... 12
3. Diagrama de classes ..... 13
   3.1. Classes principals per capa ..... 14
   3.2. Decisions de disseny ..... 14
4. Arquitectura i implementación ..... 14
   4.1. Capa Presentation ..... 14
   4.2. Capa Business ..... 14
   4.3. Capa Persistance ..... 14
   4.4. Fitxer config.json ..... 14
5. Base de dades ..... 14
6. Funcionalitats implementades ..... 14
   6.1. Registre d'usuari ..... 14
   6.2. Login d'usuari i administrador ..... 14
   6.3. Logout i eliminació de compte ..... 14
   6.4. Gestió de places ..... 14
   6.5. Consulta de l'estat del parking ..... 14
   6.6. Entrada i sortida de vehicles ..... 14
   6.7. Reserves ..... 14

6.8. Simulació de trànsit ....14
6.9. Estadístiques ....14
7. Proves i correcció d'errors detectats ....14
8. Metodologia de desenvolupament ....14
8.1. Distribució dels Sprints ....14
8.1. Eines utilitzades ....14
9. Dedicació ....14
10. Conclusions ....14
10.1. Conclusions tècniques ....14
10.2. Conclusions personals ....14
11. Bibliografia consultada ....14
12. Annexos ....14
12.1. Estructura del projecte ....14
12.2. Instruccions d'execució ....14
12.3. Usuaris de prova ....14

# 1. Resum de les especificacions del projecte

El projecte consisteix en el desenvolupament d'una aplicació d'escriptori per a la gestió d'un aparcament. L'aplicació permet diferenciar entre usuaris registrats i usuari administrador. Els usuaris normals poden registrar-se, iniciar sessió, consultar l'estat actual del parking, reservar places i registrar l'entrada o la sortida dels seus vehicles. L'administrador disposa de funcionalitats específiques per gestionar les places de l'aparcament i les reserves del sistema.

El sistema utilitza una base de dades MySQL per emmagatzemar la informació persistent dels usuaris, vehicles, places, reserves i notificacions de cancel·lació. La connexió a la base de dades es configura mitjançant el fitxer config.json, on també es defineixen valors com la contrasenya de l'administrador i el retard màxim utilitzat per la simulació de trànsit.

L'aplicació s'ha desenvolupat en Java utilitzant Swing/AWT per a la interfície gràfica. El projecte segueix una arquitectura per capes amb separació entre Presentation, Business i Persistance. Aquesta organització facilita la lectura del codi, separa responsabilitats i permet que la interfície gràfica no accedeixi directament a la base de dades.

A més de les funcionalitats principals, el projecte incorpora una simulació de trànsit de vehicles no registrats i una pantalla d'estadístiques que mostra l'evolució de l'ocupació del parking. Durant la fase final també s'han corregit errors de coherència de dades relacionats amb la propietat de les matrícules, vehicles aparcats dues vegades i places que quedaven ocupades després d'eliminar un usuari.

# 2. Disseny de la interfície gràfica

La interfície gràfica s'ha desenvolupat amb Java Swing. S'ha optat per un disseny senzill, funcional i centrat en les operacions principals del sistema. Cada pantalla correspon a una funcionalitat concreta i la navegació es gestiona des d'un controlador principal que canvia entre vistes segons l'acció de l'usuari.

## 2.1. Criteris generals de disseny

El disseny visual utilitza finestres amb títols clars, formularis centrats i botons grans per facilitar la navegació. Les dades tabulars, com l'estat del parking o les reserves, es mostren amb JTable perquè l'usuari pugui consultar la informació d'una manera ordenada. Les accions importants mostren missatges informatius o d'error mitjançant JOptionPane.

| **Component Swing/AWT** | **Ús dins del projecte** |
|---|---|
| JFrame | Finestra principal de l'aplicació. |
| JPanel | Contenidor de les diferents pantalles. |
| CardLayout | Canvi entre vistes sense obrir finestres noves. |
| JButton | Accions de login, registre, entrada, sortida, reserva i navegació. |
| JTextField | Introducció de dades com username, email, matrícula o codi de plaça. |
| JPasswordField | Introducció de la contrasenya ocultant-ne el contingut. |
| JComboBox | Selecció del tipus de vehicle o de places disponibles. |
| JTable | Visualització de l'estat del parking, reserves i gestió administrativa. |

| | |
|---|---|
| JOptionPane | Missatges d'error, confirmació i avisos a l'usuari. |

## 2.2. Pantalla de login

La pantalla d'inici de sessió permet accedir al sistema utilitzant el nom d'usuari i la contrasenya. El camp de contrasenya està implementat amb JPasswordField per mantenir-la oculta. Si l'usuari introdueix credencials incorrectes o deixa camps buits, el sistema mostra un missatge d'error.

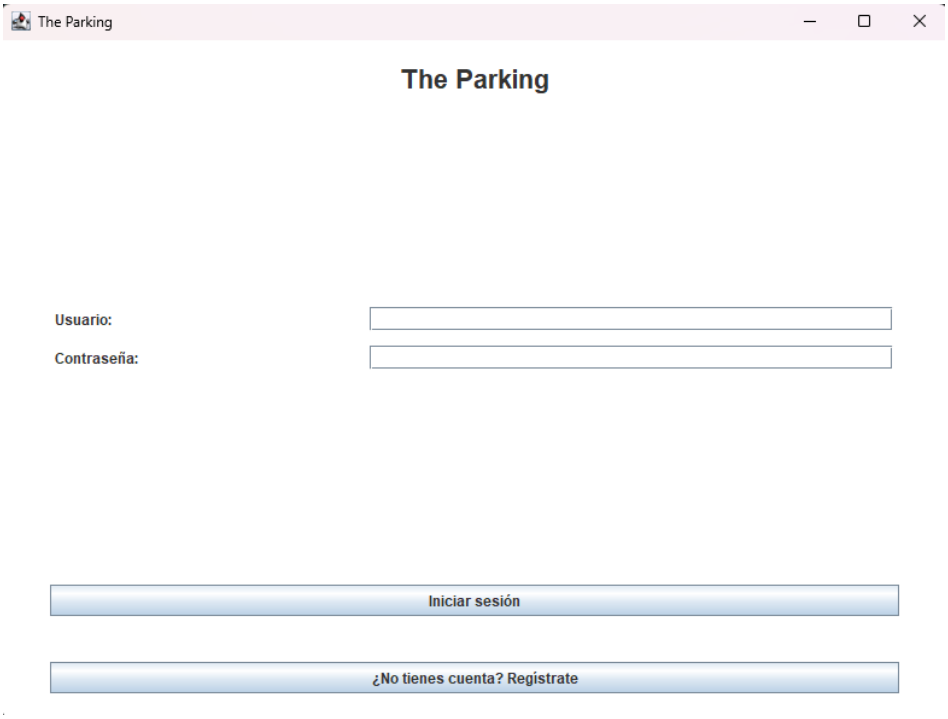



*Figura 1. Pantalla de login.*

## 2.3. Pantalla de registre

La pantalla de registre permet crear un usuari nou introduint username, email, password i confirmació de password. El controlador valida que tots els camps estiguin emplenats, que les dues contrasenyes coincideixin, que la contrasenya compleixi els requisits mínims i que el username o email no existeixin prèviament a la base de dades.

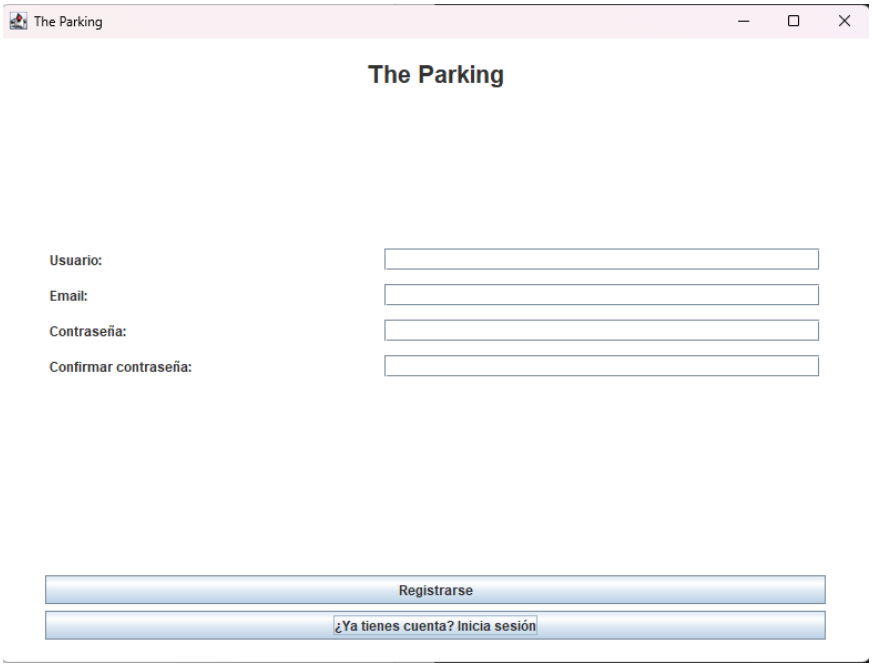



*Figura 2. Pantalla de registre d'usuari.*

## 2.4. Menú principal d'usuari normal

Quan inicia sessió un usuari normal, el menú principal mostra únicament les opcions pròpies d'aquest rol. L'usuari pot consultar l'estat del parking, reservar places, gestionar les seves reserves, registrar l'entrada o sortida del seu vehicle i accedir a l'opció de logout o eliminació de compte.



*Figura 3. Menú principal d'un usuari registrat.*

## 2.5. Menú principal d'administrador

L'usuari administrador disposa d'un menú diferenciat. Aquest rol pot accedir a la gestió de places, l'estat general del parking, la gestió de reserves i les estadístiques. En canvi, no se li mostren opcions pròpies d'un usuari normal com registrar l'entrada o sortida d'un vehicle particular.




*Figura 4. Menú principal de l'administrador.*

## 2.6. Estat actual del parking

La vista d'estat del parking mostra totes les places disponibles mitjançant una taula. Les columnes principals són el codi de la plaça, la planta, l'estat d'ocupació, l'estat de reserva, la matrícula aparcada i la matrícula reservada. Aquesta vista permet comprovar ràpidament quines places estan lliures, ocupades o reservades.

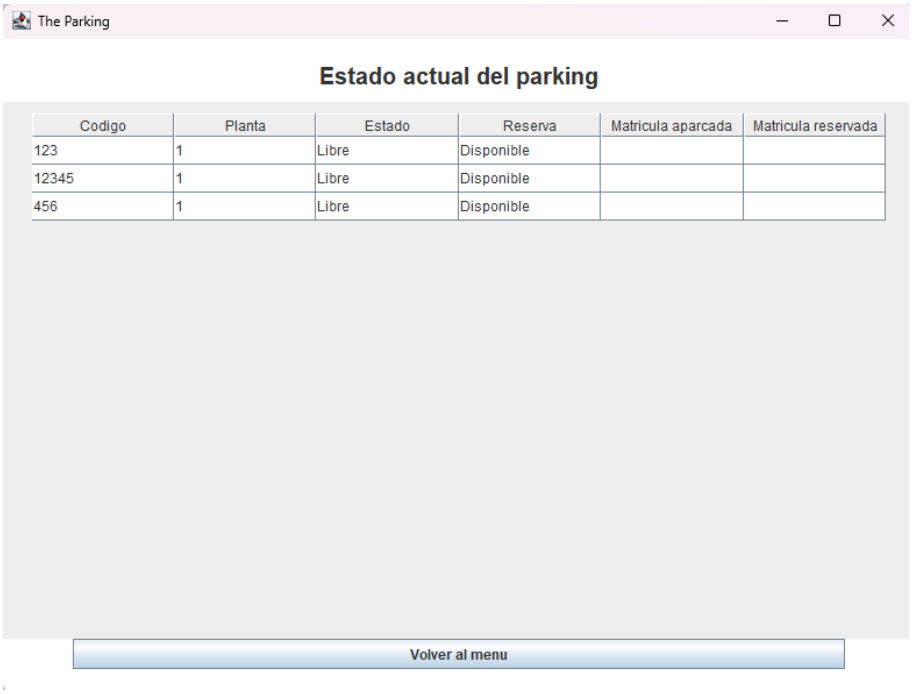



*Figura 5. Estat actual del parking.*

## 2.7. Entrada i sortida del parking

Aquesta pantalla permet a l'usuari a registrar l'entrada i la sortida d'un vehicle. L'usuari introdueix la matrícula i selecciona el tipus de vehicle. En registrar l'entrada, el sistema comprova si la matrícula ja existeix, si pertany a l'usuari connectat, si el vehicle ja està aparcat i si hi ha una plaça compatible disponible. En registrar la sortida, el sistema comprova que el vehicle existeixi, que pertanyi a l'usuari connectat i que estigui aparcat.




*Figura 6. Pantalla d'entrada i sortida de vehicles.*

## 2.8. Gestió de reserves de l'usuari

La funcionalitat de reserves permet a l'usuari crear una reserva indicant matrícula, tipus de vehicle i plaça. El sistema comprova que la plaça existeixi, que no estigui ocupada, que no tingui una reserva activa i que el tipus de vehicle coincideixi amb el tipus admès per la plaça. L'usuari també pot consultar i cancel·lar les seves reserves actives.

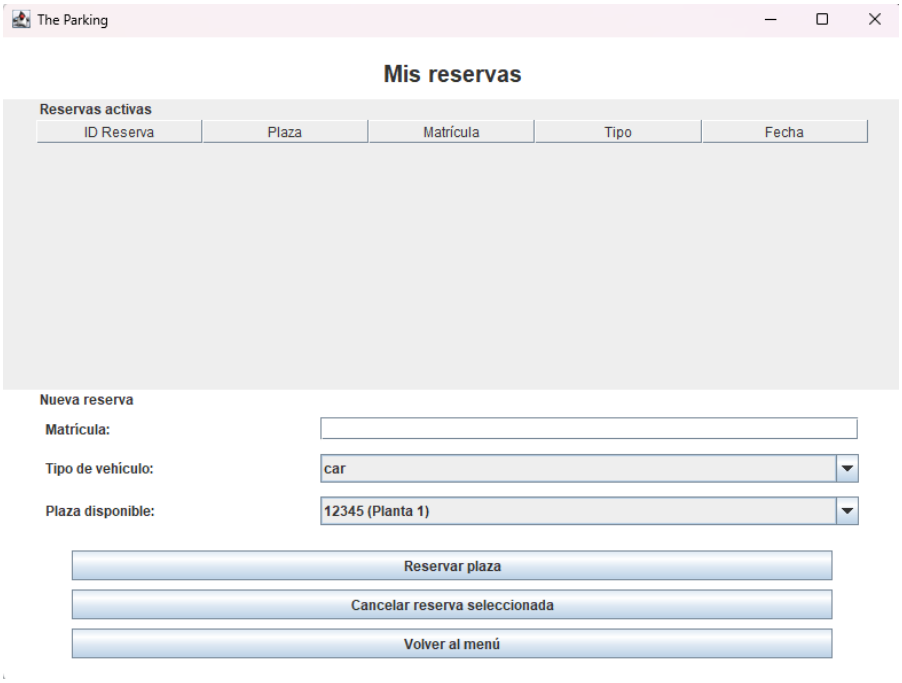



*Figura 7. Gestió de reserves de l'usuari.*

## 2.9. Gestió de places de l'administrador

L'administrador pot afegir, editar i eliminar places del parking. En crear o editar una plaça, es valida el codi, la planta i el tipus de vehicle. En eliminar una plaça, el sistema comprova que la plaça no estigui ocupada. Si la plaça té una reserva activa, el sistema intenta traspassar-la a una plaça equivalent disponible.




*Figura 8. Gestió de places per part de l'administrador.*

## 2.10. Gestió de reserves per part de l'administrador

L'administrador pot consultar totes les reserves actives del sistema amb informació de la plaça, matrícula, usuari, email i data. Si cancel·la una reserva, el sistema busca una plaça equivalent per reassignar-la i crea una notificació perquè l'usuari sigui informat quan torni a iniciar sessió.




*Figura 9. Gestió administrativa de reserves.*

## 2.11. Estadístiques

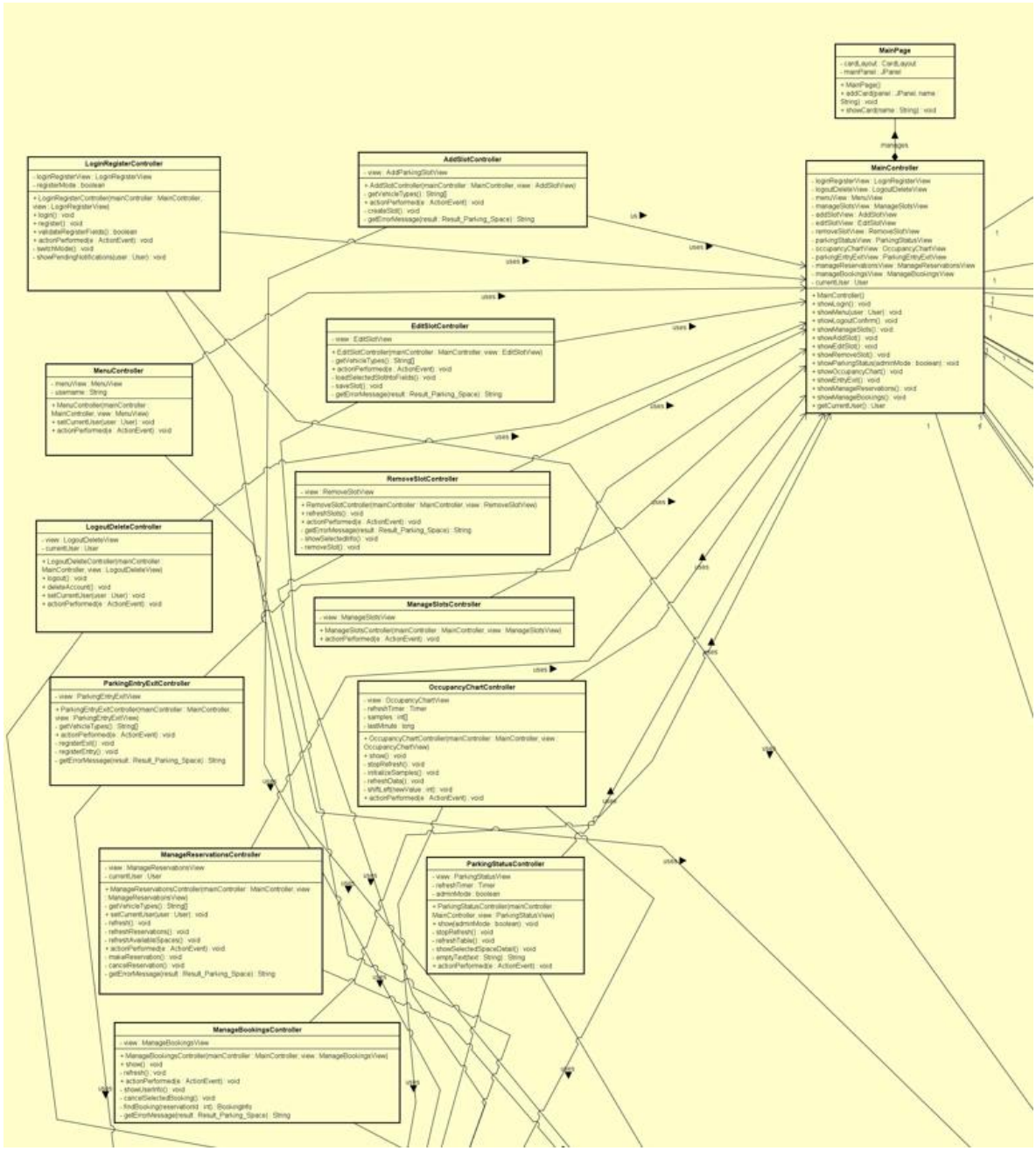
La vista d'estadístiques mostra gràficament l'ocupació del parking. S'ha implementat un gràfic de barres amb components Swing/AWT per representar el nombre de places ocupades en diferents instants. Aquesta pantalla ajuda a interpretar l'evolució de l'ocupació del sistema.

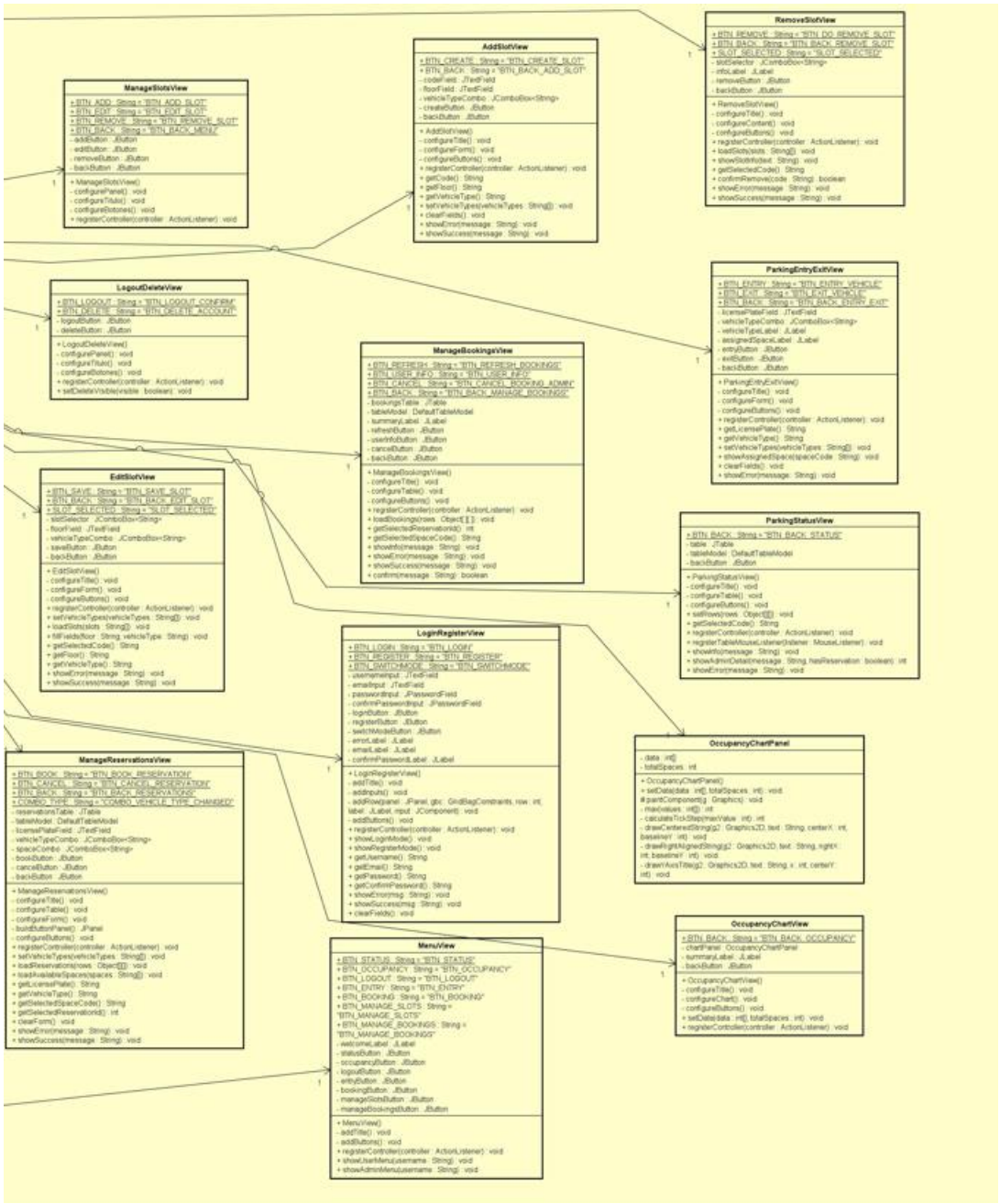


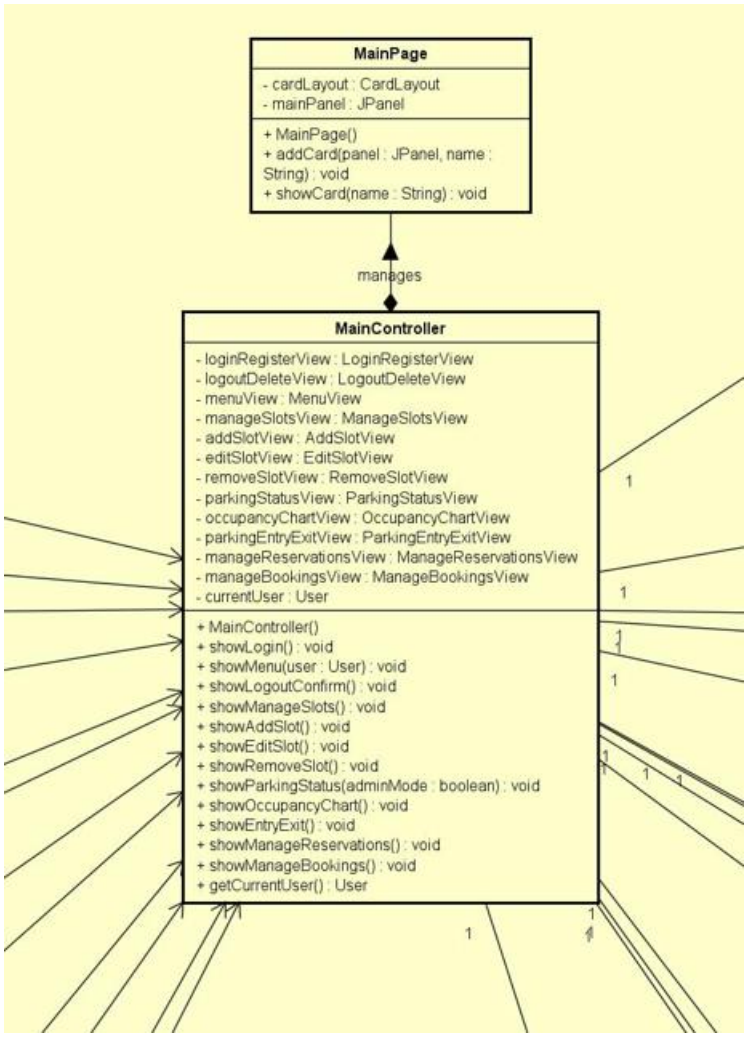
*Figura 10. Estadístiques d'ocupació del parking*

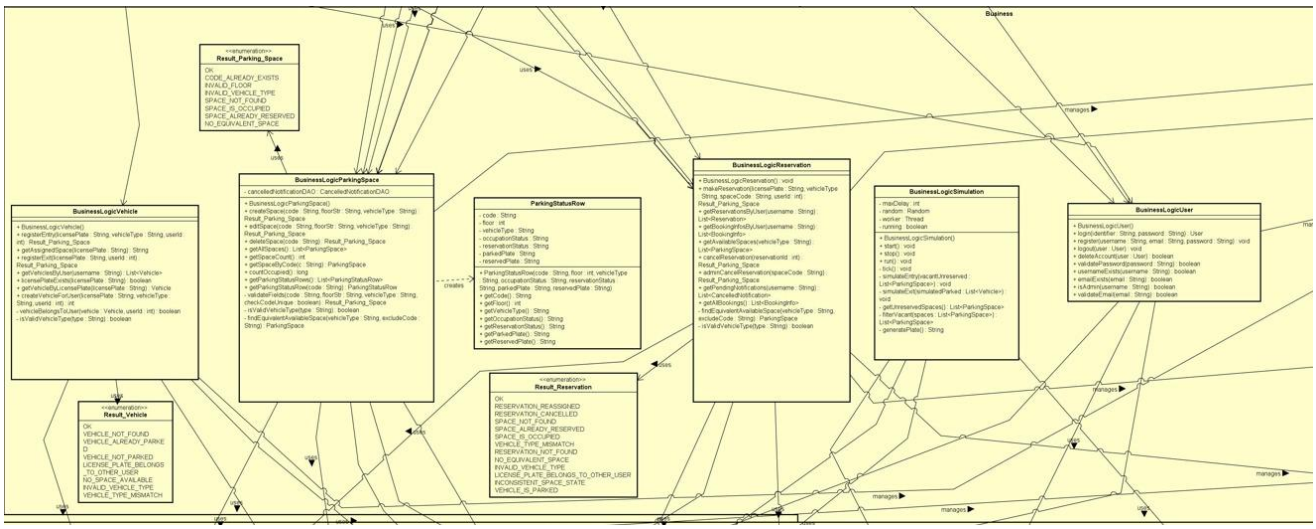
# 3. Diagrama de classes

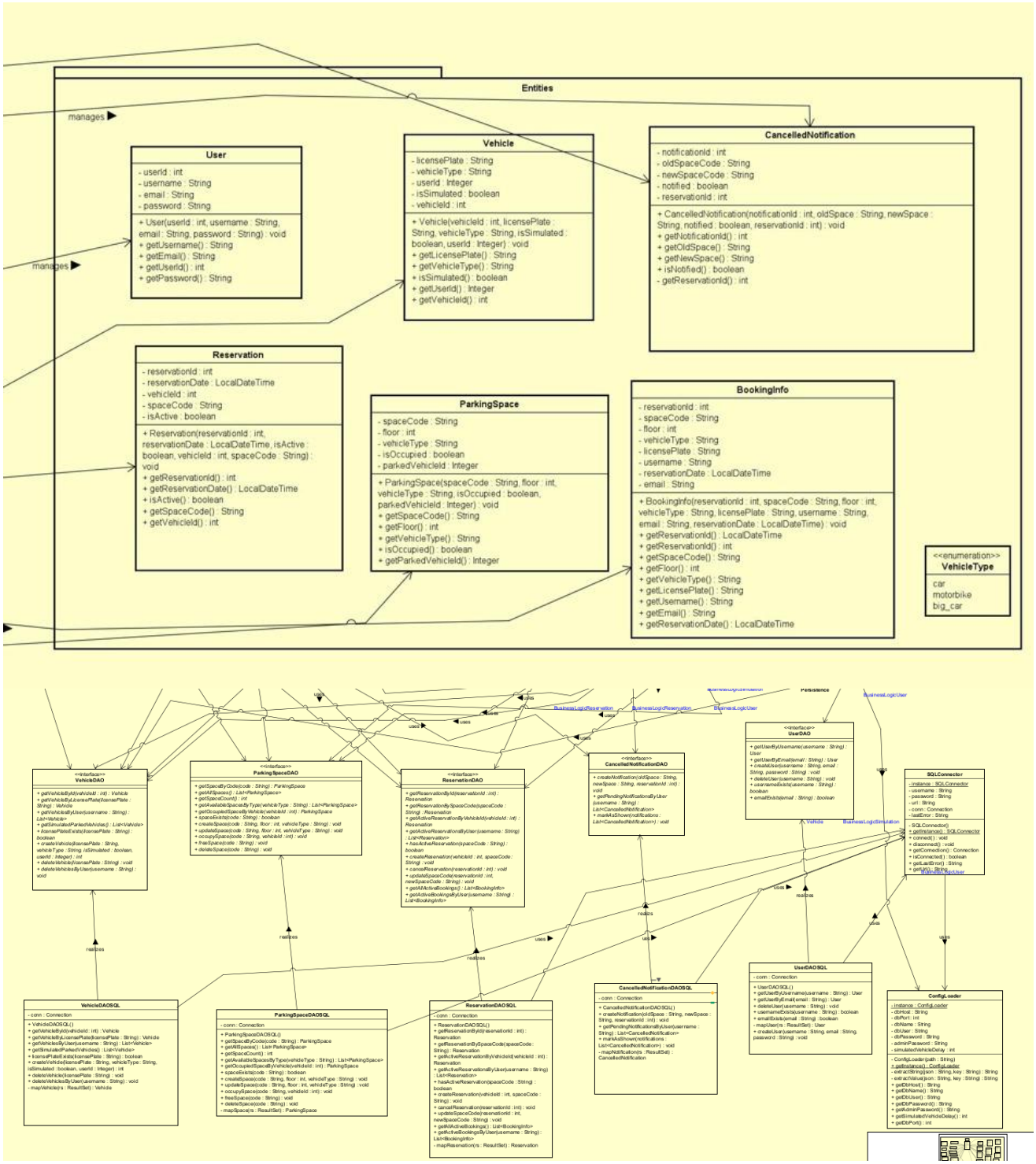
El projecte s'ha estructurat seguint una arquitectura per capes. La capa Presentation conté les vistes i controladors encarregats de gestionar la interacció amb l'usuari. La capa Business conté les classes de lògica de negoci, on s'apliquen les validacions i regles principals del sistema. La capa Persistance conté els DAO i les entitats utilitzades per accedir a la base de dades.


MainPage
- cardLayout : CardLayout
- mainPanel : JPanel
+ MainPage()
+ addCard(panel : JPanel, name : String) : void
+ showCard(name : String) : void
manages
MainController
- loginRegisterView : LoginRegisterView
- logoutDeleteView : LogoutDeleteView
- menuView : MenuView
- manageSlotsView : ManageSlotsView
- addSlotView : AddSlotView
- editSlotView : EditSlotView
- removeSlotView : RemoveSlotView
- parkingStatusView : ParkingStatusView
- occupancyChartView : OccupancyChartView
- parkingEntryExitView : ParkingEntryExitView
- manageReservationsView : ManageReservationsView
- manageBookingsView : ManageBookingsView
- currentUser : User
+ MainController()
+ showLogin() : void
+ showMenu(user : User) : void
+ showLogoutConfirm() : void
+ showManageSlots() : void
+ showAddSlot() : void
+ showEditSlot() : void
+ showRemoveSlot() : void
+ showParkingStatus(adminMode : boolean) : void
+ showOccupancyChart() : void
+ showEntryExit() : void
+ showManageReservations() : void
+ showManageBookings() : void
+ getCurrentUser() : User
LoginRegisterController
- loginRegisterView : LoginRegisterView
- registerMode : boolean
+ LoginRegisterController(mainController : MainController, view : LoginRegisterView)
+ login() : void
+ register() : void
+ validateRegisterFields() : boolean
+ actionPerformed(e : ActionEvent) : void
- switchMode() : void
- showPendingNotifications(user : User) : void
AddSlotController
- view : AddParkingSlotView
+ AddSlotController(mainController : MainController, view : AddSlotView)
- getVehicleTypes() : String[]
+ actionPerformed(e : ActionEvent) : void
- createSlot() : void
- getErrorMessage(result : Result_Parking_Space) : String
EditSlotController
- view : EditSlotView
+ EditSlotController(mainController : MainController, view : EditSlotView)
- getVehicleTypes() : String[]
+ actionPerformed(e : ActionEvent) : void
- loadSelectedSlotIntoFields() : void
- saveSlot() : void
- getErrorMessage(result : Result_Parking_Space) : String
MenuController
- menuView : MenuView
- username : String
+ MenuController(mainController : MainController, view : MenuView)
+ setCurrentUser(user : User) : void
+ actionPerformed(e : ActionEvent) : void
RemoveSlotController
- view : RemoveSlotView
+ RemoveSlotController(mainController : MainController, view : RemoveSlotView)
+ refreshSlots() : void
+ actionPerformed(e : ActionEvent) : void
- getErrorMessage(result : Result_Parking_Space) : String
- showSelectedInfo() : void
- removeSlot() : void
LogoutDeleteController
- view : LogoutDeleteView
- currentUser : User
+ LogoutDeleteController(mainController : MainController, view : LogoutDeleteView)
+ logout() : void
+ deleteAccount() : void
+ setCurrentUser(user : User) : void
+ actionPerformed(e : ActionEvent) : void
ManageSlotsController
- view : ManageSlotsView
+ ManageSlotsController(mainController : MainController, view : ManageSlotsView)
+ actionPerformed(e : ActionEvent) : void
ParkingEntryExitController
- view : ParkingEntryExitView
+ ParkingEntryExitController(mainController : MainController, view : ParkingEntryExitView)
- getVehicleTypes() : String[]
+ actionPerformed(e : ActionEvent) : void
- registerEntry() : void
- registerExit() : void
- getErrorMessage(result : Result_Parking_Space) : String
OccupancyChartController
- view : OccupancyChartView
- refreshTimer : Timer
- samples : int[]
- lastMinute : long
+ OccupancyChartController(mainController : MainController, view : OccupancyChartView)
+ show() : void
- stopRefresh() : void
- initializeSamples() : void
- refreshData() : void
- shiftLeft(newValue : int) : void
+ actionPerformed(e : ActionEvent) : void
ManageReservationsController
- view : ManageReservationsView
- currentUser : User
+ ManageReservationsController(mainController : MainController, view : ManageReservationsView)
- getVehicleTypes() : String[]
+ setCurrentUser(user : User) : void
- refresh() : void
- refreshReservations() : void
- refreshAvailableSpaces() : void
+ actionPerformed(e : ActionEvent) : void
- makeReservation() : void
- cancelReservation() : void
- getErrorMessage(result : Result_Parking_Space) : String
ParkingStatusController
- view : ParkingStatusView
- refreshTimer : Timer
- adminMode : boolean
+ ParkingStatusController(mainController : MainController, view : ParkingStatusView)
+ show(adminMode : boolean) : void
- stopRefresh() : void
- refreshTable() : void
- showSelectedSpaceDetail() : void
- emptyText(text : String) : String
+ actionPerformed(e : ActionEvent) : void
ManageBookingsController
- view : ManageBookingsView
+ ManageBookingsController(mainController : MainController, view : ManageBookingsView)
+ show() : void
- refresh() : void
+ actionPerformed(e : ActionEvent) : void
- showUserInfo() : void
- cancelSelectedBooking() : void
- findBooking(reservationId : int) : BookingInfo
- getErrorMessage(result : Result_Parking_Space) : String
uses


RemoveSlotView
+ BTN_REMOVE : String = "BTN_DO_REMOVE_SLOT"
+ BTN_BACK : String = "BTN_BACK_REMOVE_SLOT"
+ SLOT_SELECTED : String = "SLOT_SELECTED"
- slotSelector : JComboBox<String>
- infoLabel : JLabel
- removeButton : JButton
- backButton : JButton
+ RemoveSlotView()
- configureTitle() : void
- configureContent() : void
- configureButtons() : void
+ registerController(controller : ActionListener) : void
+ loadSlots(slots : String[]) : void
+ showSlotInfo(text : String) : void
+ getSelectedCode() : String
+ confirmRemove(code : String) : boolean
+ showError(message : String) : void
+ showSuccess(message : String) : void
AddSlotView
+ BTN_CREATE : String = "BTN_CREATE_SLOT"
+ BTN_BACK : String = "BTN_BACK_ADD_SLOT"
- codeField : JTextField
- floorField : JTextField
- vehicleTypeCombo : JComboBox<String>
- createButton : JButton
- backButton : JButton
+ AddSlotView()
- configureTitle() : void
- configureForm() : void
- configureButtons() : void
+ registerController(controller : ActionListener) : void
+ getCode() : String
+ getFloor() : String
+ getVehicleType() : String
+ setVehicleTypes(vehicleTypes : String[]) : void
+ clearFields() : void
+ showError(message : String) : void
+ showSuccess(message : String) : void
ManageSlotsView
+ BTN_ADD : String = "BTN_ADD_SLOT"
+ BTN_EDIT : String = "BTN_EDIT_SLOT"
+ BTN_REMOVE : String = "BTN_REMOVE_SLOT"
+ BTN_BACK : String = "BTN_BACK_MENU"
- addButton : JButton
- editButton : JButton
- removeButton : JButton
- backButton : JButton
+ ManageSlotsView()
- configurePanel() : void
- configureTitles() : void
- configureBotones() : void
+ registerController(controller : ActionListener) : void
LogoutDeleteView
+ BTN_LOGOUT : String = "BTN_LOGOUT_CONFIRM"
+ BTN_DELETE : String = "BTN_DELETE_ACCOUNT"
- logoutButton : JButton
- deleteButton : JButton
+ LogoutDeleteView()
- configurePanel() : void
- configureTitles() : void
- configureBotones() : void
+ registerController(controller : ActionListener) : void
+ setDeleteVisible(visible : boolean) : void
ParkingEntryExitView
+ BTN_ENTRY : String = "BTN_ENTRY_VEHICLE"
+ BTN_EXIT : String = "BTN_EXIT_VEHICLE"
+ BTN_BACK : String = "BTN_BACK_ENTRY_EXIT"
- licensePlateField : JTextField
- vehicleTypeCombo : JComboBox<String>
- vehicleTypeLabel : JLabel
- assignedSpaceLabel : JLabel
- entryButton : JButton
- exitButton : JButton
- backButton : JButton
+ ParkingEntryExitView()
- configureTitle() : void
- configureForm() : void
- configureButtons() : void
+ registerController(controller : ActionListener) : void
+ getLicensePlate() : String
+ getVehicleType() : String
+ setVehicleTypes(vehicleTypes : String[]) : void
+ showAssignedSpace(spaceCode : String) : void
+ clearFields() : void
+ showError(message : String) : void
ManageBookingsView
+ BTN_REFRESH : String = "BTN_REFRESH_BOOKINGS"
+ BTN_USER_INFO : String = "BTN_USER_INFO"
+ BTN_CANCEL : String = "BTN_CANCEL_BOOKING_ADMIN"
+ BTN_BACK : String = "BTN_BACK_MANAGE_BOOKINGS"
- bookingsTable : JTable
- tableModel : DefaultTableModel
- summaryLabel : JLabel
- refreshButton : JButton
- userInfoButton : JButton
- cancelButton : JButton
- backButton : JButton
+ ManageBookingsView()
- configureTitle() : void
- configureTable() : void
- configureButtons() : void
+ registerController(controller : ActionListener) : void
+ loadBookings(rows : Object[][]) : void
+ getSelectedReservationId() : int
+ getSelectedSpaceCode() : String
+ showInfo(message : String) : void
+ showError(message : String) : void
+ showSuccess(message : String) : void
+ confirm(message : String) : boolean
EditSlotView
+ BTN_SAVE : String = "BTN_SAVE_SLOT"
+ BTN_BACK : String = "BTN_BACK_EDIT_SLOT"
+ SLOT_SELECTED : String = "SLOT_SELECTED"
- slotSelector : JComboBox<String>
- floorField : JTextField
- vehicleTypeCombo : JComboBox<String>
- saveButton : JButton
- backButton : JButton
+ EditSlotView()
- configureTitle() : void
- configureForm() : void
- configureButtons() : void
+ registerController(controller : ActionListener) : void
+ setVehicleTypes(vehicleTypes : String[]) : void
+ loadSlots(slots : String[]) : void
+ fillFields(floor : String, vehicleType : String) : void
+ getSelectedCode() : String
+ getFloor() : String
+ getVehicleType() : String
+ showError(message : String) : void
+ showSuccess(message : String) : void
ParkingStatusView
+ BTN_BACK : String = "BTN_BACK_STATUS"
- table : JTable
- tableModel : DefaultTableModel
- backButton : JButton
+ ParkingStatusView()
- configureTitle() : void
- configureTable() : void
- configureButtons() : void
+ setRows(rows : Object[][]) : void
+ getSelectedCode() : String
+ registerController(controller : ActionListener) : void
+ registerTableMouseListener(listener : MouseListener) : void
+ showInfo(message : String) : void
+ showAdminDetail(message : String, hasReservation : boolean) : int
+ showError(message : String) : void
LoginRegisterView
+ BTN_LOGIN : String = "BTN_LOGIN"
+ BTN_REGISTER : String = "BTN_REGISTER"
+ BTN_SWITCHMODE : String = "BTN_SWITCHMODE"
- usernameInput : JTextField
- emailInput : JTextField
- passwordInput : JPasswordField
- confirmPasswordInput : JPasswordField
- loginButton : JButton
- registerButton : JButton
- switchModeButton : JButton
- emailLabel : JLabel
- confirmPasswordLabel : JLabel
+ LoginRegisterView()
- addTitle() : void
- addInputs() : void
- addRow(panel : JPanel, gbc : GridBagConstraints, row : int, label : JLabel, input : JComponent) : void
- addButtons() : void
+ registerController(controller : ActionListener) : void
+ showLoginMode() : void
+ showRegisterMode() : void
+ getUsername() : String
+ getEmail() : String
+ getPassword() : String
+ getConfirmPassword() : String
+ showError(msg : String) : void
+ showSuccess(msg : String) : void
+ clearFields() : void
OccupancyChartPanel
- data : int[]
- totalSpaces : int
+ OccupancyChartPanel()
+ setData(data : int[], totalSpaces : int) : void
# paintComponent(g : Graphics) : void
- max(values : int[]) : int
- calculateTickStep(maxValue : int) : int
- drawCenteredString(g2 : Graphics2D, text : String, centerX : int, baseline : int) : void
- drawRightAlignedString(g2 : Graphics2D, text : String, rightX : int, baseline : int) : void
- drawYAxisTitle(g2 : Graphics2D, text : String, x : int, centerY : int) : void
ManageReservationsView
+ BTN_BOOK : String = "BTN_BOOK_RESERVATION"
+ BTN_CANCEL : String = "BTN_CANCEL_RESERVATION"
+ BTN_BACK : String = "BTN_BACK_RESERVATIONS"
+ COMBO_TYPE : String = "COMBO_VEHICLE_TYPE_CHANGED"
- reservationsTable : JTable
- tableModel : DefaultTableModel
- licensePlateField : JTextField
- vehicleTypeCombo : JComboBox<String>
- spaceCombo : JComboBox<String>
- bookButton : JButton
- cancelButton : JButton
- backButton : JButton
+ ManageReservationsView()
- configureTitle() : void
- configureTable() : void
- configureForm() : void
- buildButtonPanel() : JPanel
- configureButtons() : void
+ registerController(controller : ActionListener) : void
+ setVehicleTypes(vehicleTypes : String[]) : void
+ loadReservations(rows : Object[][]) : void
+ loadAvailableSpaces(spaces : String[]) : void
+ getLicensePlate() : String
+ getVehicleType() : String
+ getSelectedSpaceCode() : String
+ getSelectedReservationId() : int
+ clearForm() : void
+ showError(message : String) : void
+ showSuccess(message : String) : void
OccupancyChartView
+ BTN_BACK : String = "BTN_BACK_OCCUPANCY"
- chartPanel : OccupancyChartPanel
- summaryLabel : JLabel
- backButton : JButton
+ OccupancyChartView()
- configureTitle() : void
- configureChart() : void
- configureButtons() : void
+ setData(data : int[], totalSpaces : int) : void
+ registerController(controller : ActionListener) : void
MenuView
+ BTN_STATUS : String = "BTN_STATUS"
+ BTN_OCCUPANCY : String = "BTN_OCCUPANCY"
+ BTN_LOGOUT : String = "BTN_LOGOUT"
+ BTN_ENTRY : String = "BTN_ENTRY"
+ BTN_BOOKING : String = "BTN_BOOKING"
+ BTN_MANAGE_SLOTS : String = "BTN_MANAGE_SLOTS"
+ BTN_MANAGE_BOOKINGS : String = "BTN_MANAGE_BOOKINGS"
- welcomeLabel : JLabel
- statusButton : JButton
- occupancyButton : JButton
- logoutButton : JButton
- entryButton : JButton
- bookingButton : JButton
- manageSlotsButton : JButton
- manageBookingsButton : JButton
+ MenuView()
- addTitle() : void
- addButtons() : void
+ registerController(controller : ActionListener) : void
+ showUserMenu(username : String) : void
+ showAdminMenu(username : String) : void


MainPage
- cardLayout : CardLayout
- mainPanel : JPanel
+ MainPage()
+ addCard(panel : JPanel, name : String) : void
+ showCard(name : String) : void
manages
MainController
- loginRegisterView : LoginRegisterView
- logoutDeleteView : LogoutDeleteView
- menuView : MenuView
- manageSlotsView : ManageSlotsView
- addSlotView : AddSlotView
- editSlotView : EditSlotView
- removeSlotView : RemoveSlotView
- parkingStatusView : ParkingStatusView
- occupancyChartView : OccupancyChartView
- parkingEntryExitView : ParkingEntryExitView
- manageReservationsView : ManageReservationsView
- manageBookingsView : ManageBookingsView
- currentUser : User
+ MainController()
+ showLogin() : void
+ showMenu(user : User) : void
+ showLogoutConfirm() : void
+ showManageSlots() : void
+ showAddSlot() : void
+ showEditSlot() : void
+ showRemoveSlot() : void
+ showParkingStatus(adminMode : boolean) : void
+ showOccupancyChart() : void
+ showEntryExit() : void
+ showManageReservations() : void
+ showManageBookings() : void
+ getCurrentUser() : User






Si necesiteu mes calitat adjuntarem el UML en el ficher d'entrega

*Figura 11. Diagrama de classes UML del projecte.*

## 3.1. Classes principals per capa

| Capa | Classes principals | Responsabilitat |
|---|---|---|
| Presentation | MainController, LoginRegisterController, MenuController, ParkingEntryExitController, ManageReservationsController, ManageBookingsController, ParkingStatusController, OccupancyChartController i les seves vistes associades | Gestionar la interacció amb l'usuari, capturar esdeveniments dels botons, mostrar missatges i actualitzar les vistes. |
| Business | BusinessLogicUser, BusinessLogicVehicle, BusinessLogicReservation, BusinessLogicParkingSpace, BusinessLogicSimulation | Aplicar regles de negoci, validacions, comprovacions de coherència i coordinació entre DAO. |
| Persistance/dao | UserDAO, VehicleDAO, ParkingSpaceDAO, ReservationDAO i implementacions SQL | Accedir a la base de dades mitjançant consultes SQL i retornar entitats al sistema. |
| Business/entity | User, Vehicle, ParkingSpace, Reservation, CancelledNotification | Representar els objectes principals del domini del parking. |

## 3.2. Decisions de disseny

- Separar la interfície gràfica de la lògica de negoci per evitar que les vistes accedeixin directament a la base de dades.
- Utilitzar classes DAO per centralitzar les operacions SQL i facilitar el manteniment del codi.
- Crear classes BusinessLogic per validar les operacions abans de modificar la base de dades.
- Fer servir entitats simples per transportar dades entre persistència, negoci i presentació.
- Diferenciar el comportament de l'administrador i dels usuaris registrats des del controlador de menú.

# 4. Arquitectura i implementación

Tot i que les instruccions demanen principalment el diagrama de classes, també és útil documentar l'arquitectura general del projecte. L'aplicació s'organitza en paquets que reflecteixen el patró MVC adaptat a una arquitectura per capes.

## 4.1. Capa Presentation

La capa Presentation conté les vistes i controladors Swing. Les vistes creen els components gràfics i exposen mètodes per llegir o actualitzar dades. Els controladors escolten les accions de l'usuari i criden la capa Business per executar les operacions. Aquesta separació evita concentrar tota la lògica dins de les finestres.

A més a més, aquesta capautilitza una estructura jeràrquica encapçalada per Main Controller. Aquest actua com a coordinador central de navegació entre vistes: conté referències a totes les vistes i controladors del sistema, però no els instancia al constructor. En el seu lloc, aplica un patró d'inicialització lazy: cada vista i el seu controlador associat es creen únicament la primera vegada que es navega cap a ells, mitjançant el patró if (view == null) { ... }, i a partir d'aleshores es reutilitzen. L'únic element inicialitzat al constructor és la vista de login, que és el punt d'entrada natural del sistema

## 4.2. Capa Business

La capa Business concentra les regles de negoci. Per exemple, BusinessLogicVehicle valida que una matrícula pertanyi a l'usuari connectat abans de registrar una entrada o sortida, comprova si el vehicle ja és a una plaça i gestiona l'assignació automàtica de plaça. BusinessLogicParkingSpace valida la creació, edició i eliminació de

places, i BusinessLogicReservation gestiona la creació, cancel·lació i reassignació de reserves, incloent-hi la generació de notificacions per a l'usuari.

### 4.3. Capa Persistance

La capa Persistance encapsula l'accés a la base de dades. SQLConnector obre la connexió a partir dels valors de config.json. Les classes DAOSQL executen les consultes SQL i retornen objectes de les entitats corresponents. Això permet que la resta del programa no hagi de treballar directament amb ResultSet o PreparedStatement.

### 4.4. Fitxer config.json

El fitxer config.json conté la configuració de connexió a la base de dades, la contrasenya de l'administrador i el retard màxim de la simulació de trànsit. Això evita haver de modificar el codi Java per canviar valors de configuració.

```
{
  "database":                                                    {
    "host":                                                  "localhost",
    "port":                                                         3306,
    "name":                                                  "parking_db",
    "user":                                                        "root",
    "password":                                                      ""
  },
  "adminPassword":                                           "admin1234",
  "simulatedVehicleDelay":                                          10
}
```

# 5. Base de dades

La base de dades és l'encarregada d'emmagatzemar tota la informació persistent de l'aplicació, garantint que les dades es conservin de manera segura i estructurada fins i tot després de tancar el sistema o reiniciar el servidor. Per a aquest projecte s'ha dissenyat una base de dades relacional anomenada parking_db, la qual actua com a nucli central de l'aplicació i permet gestionar de manera eficient tots els elements relacionats amb el funcionament del sistema d'aparcament.

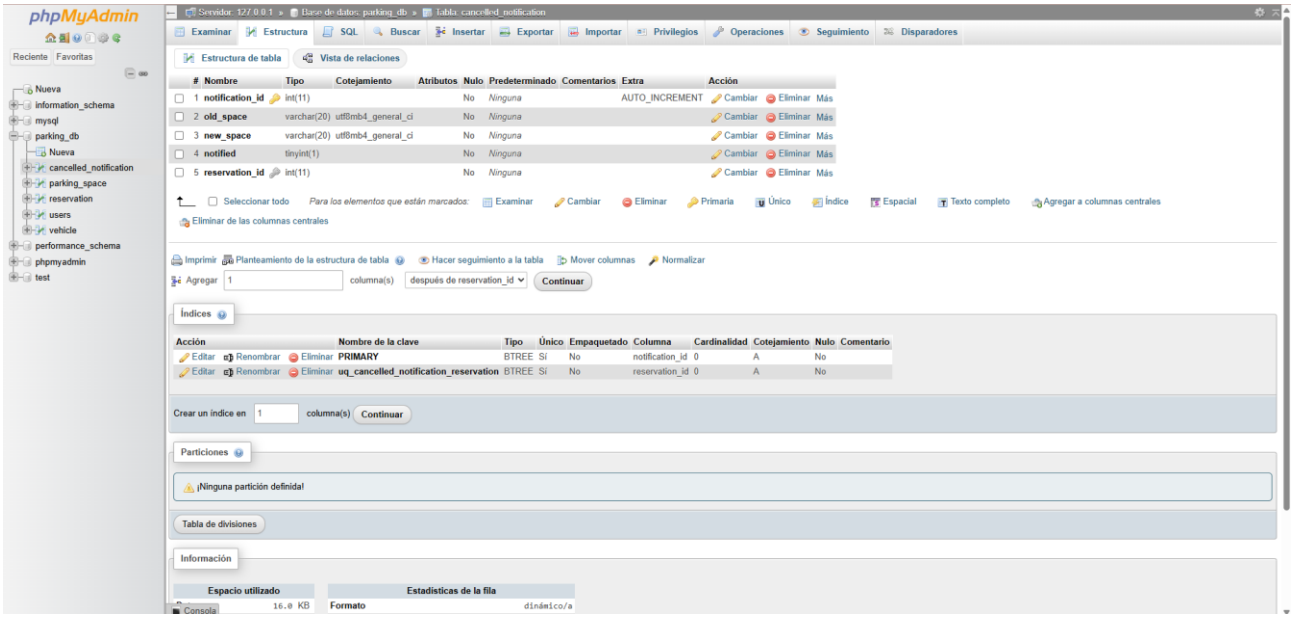



*Figura 12. Estructura principal de la base de dades.*

Dins d'aquesta base de dades s'han creat diverses taules principals que representen les diferents entitats del sistema i les seves relacions. Entre les més importants destaquen:

| **Taula** | **Finalitat** | **Camps rellevants** |
|---|---|---|
| users | Emmagatzema els usuaris registrats. | user_id, username, email, password |
| vehicle | Emmagatzema els vehicles reals i simulats. | vehicle_id, license_plate, vehicle_type, is_simulated, user_id |
| parking_space | Representa les places del parking. | space_code, floor, vehicle_type, is_occupied, parked_vehicle_id |
| reservation | Gestiona les reserves de places. | reservation_id, reservation_date, is_active, vehicle_id, space_code |
| cancelled_notification | Guarda notificacions de reserves modificades per l'administrador. | notification_id, old_space, new_space, notified, reservation_id |

Les relacions principals permeten associar vehicles a usuaris, reserves a vehicles i places, i places ocupades a vehicles aparcats. També s'han definit restriccions d'unicitat com la matrícula única del vehicle i la relació entre plaça i vehicle aparcat.

L'estructura de la base de dades s'ha dissenyat seguint criteris d'organització i eficiència, establint relacions entre les diferents taules mitjançant claus primàries i claus foranes. Això permet mantenir la integritat de les dades i assegurar que tota la informació del sistema sigui coherent i consistent. A més, aquest disseny facilita futures ampliacions de l'aplicació i millora la mantenibilitat del projecte.

# 6. Funcionalitats implementades

## 6.1. Registre d'usuari

El sistema ofereix un mòdul d'alta per a nous usuaris que interacciona directament amb la interfície gràfica i la capa de negoci. En aquesta fase, s'executen validacions estrictes abans de consolidar el registre a la base de dades:

- Unicitat de credencials: La capa Business comprova mitjançant el UserDAO que ni el username ni l'adreça de correu electrònic (email) existissin prèviament en el sistema.

- Polítiques de seguretat: Es valida que el format del correu sigui correcte, que la contrasenya compleixi els requisits de longitud i complexitat (mínim 8 caràcters amb majúscules, minúscules i números), i que coincideixi perfectament amb el camp de confirmació.

Una vegada el registre es completa de manera satisfactòria, les dades es persisteixen a la taula users, l'usuari queda autenticat automàticament i se li concedeix l'accés directament al menú principal de l'aplicació.

## 6.2. Login d'usuari i administrador

L'inici de sessió unificat centralitza l'accés en un únic panell gràfic per als dos rols del sistema. Aquest control implementa una lògica de bifurcació segons el tipus d'usuari:

- Usuari Administrador: S'identifica de forma fixa amb el mot clau "admin". El sistema no valida aquesta credencial contra la base de dades relacional, sinó que comprova la seva contrasenya comparant-la directament amb el valor assignat al camp adminPassword del fitxer de configuració local config.json. En cas d'èxit, es renderitza el menú administratiu.

- Usuaris Normals (Registrats): El sistema proporciona la flexibilitat d'entrar introduint indistintament el nom d'usuari o el correu electrònic. La capa de negoci delega la consulta a la persistència, verificant que la contrasenya introduïda coincideixi amb la de la taula users.

Si algun dels camps es deixa buit o les credencials són errònies, el controlador captura l'anomalia i mostra un missatge genèric d'error mitjançant un diàleg visual d'alerta.

## 6.3. Logout i eliminació de compte

L'aplicació garanteix el control total del cicle de vida de la sessió i el compliment de la normativa vigent de protecció de dades (RGPD):

- Tancar sessió (Logout): Permet a qualsevol usuari invalidar el seu estat d'autenticació actiu des de qualsevol punt de la interfície. En executar-se, el controlador principal neteja les dades de l'usuari en memòria i reconfigura el disseny de la finestra (CardLayout) per fer retornar l'usuari a la pantalla inicial de login o registre.

- Eliminació del compte: Acció crítica que suprimeix per complet tota la traçabilitat de l'usuari en el sistema (base de dades, memòria RAM i fitxers). Per evitar corrupcions en l'estat operatiu del pàrquing, la classe BusinessLogicUser executa de manera prèvia un protocol de neteja: es localitzen tots els vehicles associats a l'usuari, s'alliberen immediatament les places que aquests poguessin estar upant en aquell moment i es cancel·len les seves reserves actives. Un cop el recinte queda lliure d'incoherències, es procedeix a l'esborrat físic del registre a la base de dades.

## 6.4. Gestió de places

Mòdul operatiu d'ús exclusiu per a l'usuari administrador que permet controlar l'arquitectura i cabuda de l'aparcament mitjançant mètodes CRUD:

- Creació i edició: L'administrador pot donar d'alta noves places o modificar les existents especificant el codi identificador únic, el pis o planta i el tipus de vehicle admès (moto, cotxe, cotxe gran). La capa de negoci valida que el codi no estigui repetit, que el pis sigui coherent i que el vehicle es trobi homologat pel sistema.

- Eliminació: Abans de suprimir una plaça de la taula parking_space, la lògica de negoci comprova que no hi hagi cap vehicle estacionat físicament en ella. Si la plaça es troba lliure de vehicles però està assignada a una reserva activa d'un usuari, el sistema executa de forma automàtica una cerca d'una plaça equivalent

disponible (mateixa planta i mateix tipus de vehicle) per traspassar-li els drets de reserva. Si no es troba cap alternativa similar, s'elimina la reserva del sistema per complet.

## 6.5. Consulta de l'estat del parking

Aquesta funcionalitat actua com el monitor central d'informació en temps real per a ambdós rols de l'aplicació. Mitjançant una graella estructural basada en el component JTable, es llegeix l'estat actual del recinte directament des de la base de dades, exposant de forma clara les següents columnes:

- El codi únic de la plaça i la planta on es localitza.

- L'estat d'ocupació (si la plaça té un vehicle aparcat físicament en aquell instant o si està buida).

- L'estat de la reserva (si la plaça està lliure de compromisos o si ha estat reservada per a un ús futur).

- La matrícula del vehicle associat, diferenciant de manera estricta entre la matrícula que està actualment estacionada i la matrícula que té la plaça reservada al seu nom.

## 6.6. Entrada i sortida de vehicles

Els usuaris registrats disposen de components visuals per notificar el moviment físic dels seus vehicles introduint exclusivament la matrícula. El flux lògic de control es gestiona a la capa de negoci:

- Registre d'Entrada: El sistema avalua si la matrícula compta amb una reserva activa sobre una plaça concreta. En cas afirmatiu, se li assigna automàticament aquella plaça i es commuta el seu estat a "ocupada". Si el vehicle no té reserva, el sistema demana definir el tipus de vehicle, comprova que la matrícula pertanyi realment a l'usuari en sessió, verifica que no estigui ja dins del pàrquing i busca una plaça lliure i compatible que no estigui reservada per ningú, indicant-li a l'usuari el codi on ha d'estacionar.

- Registre de Sortida: El sistema valida l'existència de la matrícula, confirma la propietat del vehicle vinculada a l'usuari connectat i comprova que estigui realment aparcat. En processar la sortida, s'allibera immediatament la cel·la a la taula parking_space, deixant-la disponible per a futurs moviments de trànsit.

## 6.7. Reserves

Aquesta funcionalitat permet als usuaris assegurar-se una plaça de forma anticipada i als administradors auditar el funcionament global del sistema.

- Perspectiva de l'Usuari: L'usuari pot formalitzar una reserva aportant la matrícula i el tipus de vehicle. L'aplicació filtra i mostra a la interfície només aquelles places del catàleg que són compatibles amb el tipus de cotxe i que no estan ni ocupades ni reservades prèviament per tercers. L'usuari disposa d'un historial personal on pot consultar les seves reserves actives o donar-les de baixa confirmant la matrícula.

- Perspectiva de l'Administrador: L'administrador disposa d'una vista global de supervisió on pot veure les dades completes de totes les reserves: codi de plaça, matrícula, identitat de l'usuari, correu i data de creació. L'administrador té el poder de cancel·lar de forma unilateral qualsevol reserva. Quan ho fa, el sistema busca

reassignar-la de manera automàtica a una altra plaça equivalent i registra una alerta persistent a la taula cancelled_notification. La propera vegada que l'usuari afectat inicií sessió, el controlador detectarà el registre pendent i li mostrarà un missatge emergent informatiu.

### 6.8. Simulació de trànsit

Per validar la resiliència, l'escalabilitat i el correcte comportament de l'aplicació en un escenari real massiu, s'ha implementat un mòdul que automatitza el trànsit de vehicles que entren i surten sense reserves prèvies.

Aquesta simulació s'executa en segon pla mitjançant un fil de treball independent (Thread) que implementa la interfície Runnable. La periodicitat de les accions és completament dinàmica: el fil es llegeix el valor numèric en segons configurat a simulatedVehicleDelay dins del fitxer JSON i calcula un retard pseudoaleatori entre 1 i aquest límit abans de llançar la següent acció.

L'elecció de l'operació (si es simula una entrada o una sortida) es determina matemàticament mitjançant un càlcul de probabilitats basat en el nivell d'ocupació real, excloent de l'equació totes les places que tenen una reserva en actiu: (FOTO DE LA FORMULA)

A mesura que l'aparcament està més buit, la probabilitat d'entrada és màxima. Quan un vehicle simulat entra, es genera a l'atzar una matrícula única, s'insereix el registre amb el flag is_simulated = true i se li assigna una de les cel·les buides compatibles. Si el sistema determina una sortida, es tria a l'atzar un dels cotxes simulat que es trobaven prèviament allotjats al pàrquing i s'allibera la seva plaça de forma automàtica.

### 6.9. Estadístiques

Amb l'objectiu d'analitzar de manera analítica l'evolució temporal de la confluència de vehicles dins del recinte, s'ha dissenyat un panell dedicat a les estadístiques d'ocupació. Aquesta pantalla renderitza un gràfic de barres dinàmic que mostra de manera cronològica el volum històric de places que han estat retingudes per vehicles al llarg de l'última hora d'activitat.

Per complir estrictament amb els requisits de la pràctica de no utilitzar cap llibreria de tercers, aquest component gràfic s'ha programat de manera nativa sobreescrivint el mètode de dibuix primitiu de Swing/AWT, gestionant manualment el càlcul de coordenades lineals, la representació geomètrica de cadascun dels rectangles de dades i l'escalat proporcional dels eixos cartesianus de la interfície en funció del volum màxim de places disponibles del pàrquing. L'historial emmagatzema les dades minut a minut, desplaçant la informació i actualitzant la barra de l'extrem d'acord amb els esdeveniments en temps real del sistema

## 7. Proves i correcció d'errors detectats

Durant el desenvolupament de les diferents fases, la implementació d'una arquitectura de programari per capes va permetre utilitzar la fase de proves de consistència per a aïllar i depurar errors logicials complexos. Molts d'aquests errors d'integritat no provocaven fallades de compilació ni aturaven l'execució de la màquina

virtual de Java, però sí que violaven les regles operatives del negoci de l'aparcament i podien comprometre la fiabilitat de la base de dades.

Tots aquests errors estructurals van ser corregits directament dins de les classes del paquet de lògica de negoci (BusinessLayer), que és l'estrat correcte i dissenyat per aplicar els protocols de consistència de dades. A continuació, es detalla la matriu de control d'errors auditat durant el projecte:

| **Error detectat** | **Causa** | **Solució aplicada** |
|---|---|---|
| Un usuari podia treure un vehicle d'un altre usuari. | La sortida només buscava el vehicle per matrícula i no comprovava el propietari. | registerExit rep el userId de l'usuari connectat i comprova que el vehicle li pertanyi. |
| Es podia aparcar una matrícula que ja estava aparcada. | En entrar, el sistema no comprovava si el vehicle ja ocupava una plaça. | registerEntry comprova si findOccupiedSpaceByVehicle retorna una plaça ocupada i bloqueja l'operació. |
| Els vehicles nous es podien crear sense usuari associat. | En crear vehicle es passava null com a user_id. | Ara createVehicle rep el userId de l'usuari connectat. |
| En eliminar un usuari, una plaça podia quedar com ocupada sense matrícula. | La cascada eliminava el vehicle, però is_occupied podia quedar a 1. | Abans de deleteUser, BusinessLogicUser allibera les places ocupades pels vehicles de l'usuari. |

Aquestes correccions afecten principalment els fitxers BusinessLogicVehicle.java, ParkingEntryExitController.java, MainController.java i BusinessLogicUser.java.

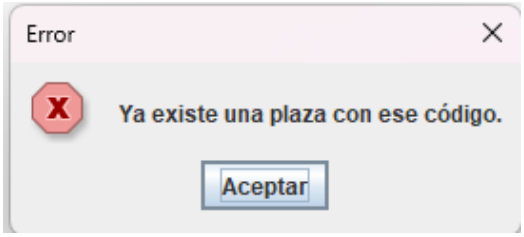



*Figura 13. Missatge d'error quan es crea un plaça amb el mateix identificador*

# 8. Metodologia de desenvolupament

El desenvolupament del projecte es va organitzar seguint una metodologia basada en funcionalitats i en una divisió del treball per sprints, amb l'objectiu de mantenir una evolució progressiva i ordenada de l'aplicació. En una primera fase es va realitzar una anàlisi detallada de l'enunciat i dels requisits del sistema, definint tant l'estructura general de l'aplicació com el model de dades necessari per al correcte funcionament del projecte.

Posteriorment, es va implementar la base del sistema, incloent-hi l'estructura principal del projecte, la connexió amb la base de dades i les funcionalitats bàsiques relacionades amb la gestió d'usuaris i vehicles. Un cop establerta aquesta base funcional, es van afegir progressivament noves característiques, com les funcionalitats específiques d'administrador, els sistemes de simulació i estadístiques, així com les proves finals i la documentació del projecte.

Aquest enfocament per fases va permetre desenvolupar i validar cada part del sistema de manera independent, facilitant la detecció d'errors, la reorganització del codi i la millora contínua de l'aplicació durant tot el procés de desenvolupament.

## 8.1. Distribució dels Sprints

Sprint 1

| Actividad | Persona asignada |
|---|---|
| DPOO25EPK1-2 Disseny UML 1 | ignasi.casanas |
| DPOO25EPK1-3 Base de dades 1 | judith.caralt |
| DPOO25EPK1-4 Disseny figma 1 | ignasi.casanas |
| DPOO25EPK1-5 Base de dades 2 | pol.martin |
| DPOO25EPK1-6 Disseny UML 2 | cristina.ponsa |
| DPOO25EPK1-7 Disseny UML 3 | aras.mora |
| DPOO25EPK1-8 Disseny figma 2 | cristina.ponsa |
| DPOO25EPK1-9 Disseny figma 3 | aras.mora |

Sprint 2

| Actividad | Persona asignada |
|---|---|
| DPOO25EPK1-67 Tests de lógica (ST10) | aras.mora |
| DPOO25EPK1-66 Gestión de reservas al eliminar (ST10) | aras.mora |
| DPOO25EPK1-65 CRUD de plazas (ST10) | cristina.ponsa |
| DPOO25EPK1-64 Validar planta numérica (ST10) | cristina.ponsa |
| DPOO25EPK1-63 Validar tipo vehículo (ST10) | cristina.ponsa |
| DPOO25EPK1-62 Validar ID único (ST10) | aras.mora |
| DPOO25EPK1-61 Tests UI (ST9) | aras.mora |
| DPOO25EPK1-60 Validaciones UI (ST9) | aras.mora |
| DPOO25EPK1-59 Botón eliminar plaza (ST9) | ignasi.casanas |
| DPOO25EPK1-58 Formulario editar plaza (ST9) | judith.caralt |
| DPOO25EPK1-57 Formulario crear plaza (ST9) | cristina.ponsa |
| DPOO25EPK1-56 Crear vista de gestión de plazas (ST9) | pol.martin |
| DPOO25EPK1-55 Tests de eliminación completa (ST8) | aras.mora |
| DPOO25EPK1-54 Bloquear eliminación de admin (ST8) | cristina.ponsa |
| DPOO25EPK1-53 Eliminar archivos locales si existen (ST8) | cristina.ponsa |
| DPOO25EPK1-52 Eliminar datos en memoria (ST8) | cristina.ponsa |

| | | |
|---|---|---|
| ☐ | DPOO25EPK1-51 Eliminar usuario de DB (ST8) | aras.mora |
| ☐ | DPOO25EPK1-50 Tests de seguridad (ST7) | aras.mora |
| ☐ | DPOO25EPK1-49 Invalidar tokens/sesión (ST7) | aras.mora |
| ☐ | DPOO25EPK1-48 Borrar datos en memoria (RAM) (ST7) | aras.mora |
| ☐ | DPOO25EPK1-47 Implementar limpieza de sesión (ST7) | judith.caralt |
| ☐ | DPOO25EPK1-46 Tests UI (ST6) | pol.martin |
| ☐ | DPOO25EPK1-45 Redirección a login (ST6) | cristina.ponsa |
| ☐ | DPOO25EPK1-44 Confirmación de borrado (ST6) | aras.mora |
| ☐ | DPOO25EPK1-43 Mostrar en todas las vistas (ST6) | ignasi.casanas |
| ☐ | DPOO25EPK1-42 Añadir botón delete account (ST6) | ignasi.casanas |
| ☐ | DPOO25EPK1-41 Añadir botón logout global (ST6ç) | ignasi.casanas |
| ☐ | DPOO25EPK1-40 Tests de login (ST5) | aras.mora |
| ☐ | DPOO25EPK1-39 Redirección a main view (ST5) | ignasi.casanas |
| ☐ | DPOO25EPK1-38 Crear sesión de usuario (ST5) | cristina.ponsa |
| ☐ | DPOO25EPK1-37 Control de errores (ST5) | cristina.ponsa |
| ☐ | DPOO25EPK1-36 Validar password (ST5) | cristina.ponsa |

| | | |
|---|---|---|
| ☐ | DPOO25EPK1-35 Implementar consulta de usuario en DB ... | aras.mora |
| ☐ | DPOO25EPK1-34 Tests de autenticación (ST4) | aras.mora |
| ☐ | DPOO25EPK1-33 Redirección tras login admin (ST4) | aras.mora |
| ☐ | DPOO25EPK1-32 Diferenciar rol admin vs user (ST4) | ignasi.casanas |
| ☐ | DPOO25EPK1-31 Imlementar servicio de autenticación ad... | aras.mora |
| ☐ | DPOO25EPK1-30 Leer credenciales admin desde config (... | aras.mora |
| ☐ | DPOO25EPK1-29 Tests de UI básicos (ST3) | aras.mora |
| ☐ | DPOO25EPK1-28 Manejo de errores visuales (ST3) | pol.martin |
| ☐ | DPOO25EPK1-27 Integrar con controlador (ST3) | aras.mora |
| ☐ | DPOO25EPK1-26 Validaciones de campos (ST3) | aras.mora |
| ☐ | DPOO25EPK1-25 Crear formulario (ST3) | aras.mora |
| ☐ | DPOO25EPK1-24 Diseñar pantalla login (ST3) | cristina.ponsa |
| ☐ | DPOO25EPK1-23 Tests de integración con DB (ST2) | pol.martin |
| ☐ | DPOO25EPK1-22 Manejo de errores de conexión (ST2) | judith.caralt |
| ☐ | DPOO25EPK1-21 Implementar relaciones entre tablas (ST2) | judith.caralt |
| ☐ | DPOO25EPK1-20 Crear DAOs por entidad (ST2) | pol.martin |

| | | |
|---|---|---|
| ☐ | DPOO25EPK1-19 Implementar CRUD genérico (ST2) | pol.martin |
| ☐ | DPOO25EPK1-18 Crear DAO base (ST2) | judith.caralt |
| ☐ | DPOO25EPK1-17 Configurar conexión a base de datos (ST2) | judith.caralt |
| ☐ | DPOO25EPK1-16 Tests unitarios de lectura de config (ST1) | pol.martin |
| ☐ | DPOO25EPK1-15 Exponer config a otras capas (ST1) | pol.martin |
| ☐ | DPOO25EPK1-14 Validar estructura JSON (ST1) | judith.caralt |
| ☐ | DPOO25EPK1-13 Implementar servicio de carga de config... | judith.caralt |
| ☐ | DPOO25EPK1-12 Implementar lector de config.json (ST1) | pol.martin |
| ☐ | DPOO25EPK1-11 Crear modelo de configuración (ST1) | pol.martin |
| ☐ | DPOO25EPK1-10 Programacio Pantallas Login/Register i M... | ignasi.casanas |
| ☐ | DPOO25EPK1-9 Disseny figma 3 | aras.mora |
| ☐ | DPOO25EPK1-8 Disseny figma 2 | cristina.ponsa |
| ☐ | DPOO25EPK1-7 Disseny UML 3 | aras.mora |
| ☐ | DPOO25EPK1-6 Disseny UML 2 | cristina.ponsa |
| ☐ | DPOO25EPK1-4 Disseny figma 1 | ignasi.casanas |

Sprint 3

| ☐ | Actividad | Persona asignada |
|---|---|---|
| ☐ | DPOO25EPK1-113 Tests (ST19) | pol.martin |
| ☐ | DPOO25EPK1-112 Validar tipo vehículo (ST19) | aras.mora |
| ☐ | DPOO25EPK1-111 Liberar plaza al salir (ST19) | aras.mora |
| ☐ | DPOO25EPK1-110 Asignar plaza automáticamente (ST19) | ignasi.casanas |
| ☐ | DPOO25EPK1-109 Buscar reserva (ST19) | judith.caralt |
| ☐ | DPOO25EPK1-108 Validar matrícula (ST19) | pol.martin |
| ☐ | DPOO25EPK1-107 Tests UI (ST18) | pol.martin |
| ☐ | DPOO25EPK1-106 Mensajes de confirmación (ST18) | ignasi.casanas |
| ☐ | DPOO25EPK1-105 Input matrícula (ST18) | cristina.ponsa |
| ☐ | DPOO25EPK1-104 Formulario salida (ST18) | cristina.ponsa |
| ☐ | DPOO25EPK1-103 Tests UI (ST17) | aras.mora |
| ☐ | DPOO25EPK1-102 Integrar con lógica (ST17) | aras.mora |
| ☐ | DPOO25EPK1-101 Mostrar mensajes éxito/error (ST17) | aras.mora |
| ☐ | DPOO25EPK1-100 Input matrícula (ST17) | ignasi.casanas |
| ☐ | DPOO25EPK1-99 Formulario entrada vehículo (ST17) | judith.caralt |
| ☐ | DPOO25EPK1-98 Tests (ST16) | judith.caralt |

| | | |
|---|---|---|
| ☐ | DPOO25EPK1-97 Servicio de métricas (ST16) | cristina.ponsa |
| ☐ | DPOO25EPK1-96 Actualización cada minuto (ST16) | cristina.ponsa |
| ☐ | DPOO25EPK1-95 Implementar gráfico Swing (ST16) | pol.martin |
| ☐ | DPOO25EPK1-94 Recoger datos históricos (ST16) | judith.caralt |
| ☐ | DPOO25EPK1-93 Tests (ST15) | judith.caralt |
| ☐ | DPOO25EPK1-92 Marcar como leída (ST15) | ignasi.casanas |
| ☐ | DPOO25EPK1-91 Mostrar al login (ST15) | ignasi.casanas |
| ☐ | DPOO25EPK1-90 Guardar notificación en DB (ST15) | pol.martin |
| ☐ | DPOO25EPK1-89 Sistema de notificaciones (ST15) | cristina.ponsa |
| ☐ | DPOO25EPK1-88 Tests (ST14) | ignasi.casanas |
| ☐ | DPOO25EPK1-87 Notificar usuario (ST14) | ignasi.casanas |
| ☐ | DPOO25EPK1-86 Reasignar reserva si aplica (ST14) | aras.mora |
| ☐ | DPOO25EPK1-85 Cancelar reserva (admin)(ST14) | aras.mora |
| ☐ | DPOO25EPK1-84 Obtener info de una plaza (ST14) | cristina.ponsa |
| ☐ | DPOO25EPK1-83 Tests (ST13) | aras.mora |
| ☐ | DPOO25EPK1-82 Eventos create, edit, delete, reserve (ST... | ignasi.casanas |

| | | |
|---|---|---|
| ☐ | DPOO25EPK1-81 Actualización en tiempo real (ST13) | cristina.ponsa |
| ☐ | DPOO25EPK1-80 Sincronización con DB (ST13) | pol.martin |
| ☐ | DPOO25EPK1-79 Servicio de estado global parking (ST13) | cristina.ponsa |
| ☐ | DPOO25EPK1-78 Tests UI (ST12) | ignasi.casanas |
| ☐ | DPOO25EPK1-77 Condición solo si reservada (ST12) | aras.mora |
| ☐ | DPOO25EPK1-76 Botón cancelar reserva (ST12) | ignasi.casanas |
| ☐ | DPOO25EPK1-75 Mostrar info completa (ST12) | judith.caralt |
| ☐ | DPOO25EPK1-74 Vista detalle de plaza (ST12) | judith.caralt |
| ☐ | DPOO25EPK1-73 Tests UI (ST11) | judith.caralt |
| ☐ | DPOO25EPK1-72 Click en fila, detalle (ST11) | pol.martin |
| ☐ | DPOO25EPK1-71 Actualización en tiempo real (ST11) | pol.martin |
| ☐ | DPOO25EPK1-70 Mostar columnas (ST11) | pol.martin |
| ☐ | DPOO25EPK1-69 Crear JTable de plazas (ST11) | pol.martin |

Sprint 4

| Actividad | Persona asignada |
|---|---|
| DPOO25EPK1-137 Tests (ST23) | ignasi.casanas |
| DPOO25EPK1-136 Schedule/timer (ST23) | cristina.ponsa |
| DPOO25EPK1-135 Asignar plazas (ST23) | pol.martin |
| DPOO25EPK1-134 Generar matrículas random (ST23) | aras.mora |
| DPOO25EPK1-133 Simular salida (ST23) | ignasi.casanas |
| DPOO25EPK1-132 Simular entrada (ST23) | ignasi.casanas |
| DPOO25EPK1-131 Generador random de tiempo (ST23) | judith.caralt |
| DPOO25EPK1-130 Leer delay desde config (ST23) | judith.caralt |
| DPOO25EPK1-129 Tests (ST22) | cristina.ponsa |
| DPOO25EPK1-128 Actualizar estado plaza (ST22) | cristina.ponsa |
| DPOO25EPK1-127 Cancelar reserva (ST22) | pol.martin |
| DPOO25EPK1-126 Filtrar por tipo vehículo (ST22) | pol.martin |
| DPOO25EPK1-125 Validar disponibilidad (ST22) | aras.mora |
| DPOO25EPK1-124 Crear reserva (ST22) | aras.mora |
| DPOO25EPK1-123 Tests UI (ST21) | ignasi.casanas |
| DPOO25EPK1-122 Confirmación con matrícula (ST21) | ignasi.casanas |
| DPOO25EPK1-121 Botón cancelar (ST21) | judith.caralt |
| DPOO25EPK1-120 Listar reservas usuario (ST21) | judith.caralt |
| DPOO25EPK1-119 Tests UI (ST20) | cristina.ponsa |
| DPOO25EPK1-118 Confirmación (ST20) | cristina.ponsa |
| DPOO25EPK1-117 Selección de plaza (ST20) | pol.martin |
| DPOO25EPK1-116 Mostrar plazas disponibles (ST20) | pol.martin |
| DPOO25EPK1-115 Input matrícula y tipo (ST20) | aras.mora |
| DPOO25EPK1-114 Formulario reserva (ST20) | aras.mora |

### 8.1. Eines utilitzades

| Eina | Ús |
|---|---|
| IntelliJ IDEA | Desenvolupament i execució del projecte Java. |
| Java | Llenguatge principal del projecte. |
| Swing/AWT | Construcció de la interfície gràfica. |
| MySQL / MariaDB | Sistema gestor de base de dades. |
| phpMyAdmin | Creació, importació i revisió de la base de dades. |
| JDBC | Connexió entre Java i la base de dades. |
| Javadoc | Generació de documentació HTML del codi. |
| Word / Google Docs | Redacció de la memòria del projecte. |

# 9. Dedicació

La dedicació total estimada del projecte s'ha dividit en comprensió de les especificacions, anàlisi i disseny, codificació, proves i documentació. La taula següent mostra una estimació orientativa que el grup pot ajustar segons les hores reals dedicades.

| Activitat | Hores estimades |
|---|---|
| Comprensió de les especificacions | 7h |

| | |
|---|---|
| Anàlisi i disseny | 10h |
| Codificació | 40h |
| Proves i correcció d'errors | 10h |
| Documentació | 5h |
| Total | 72h |

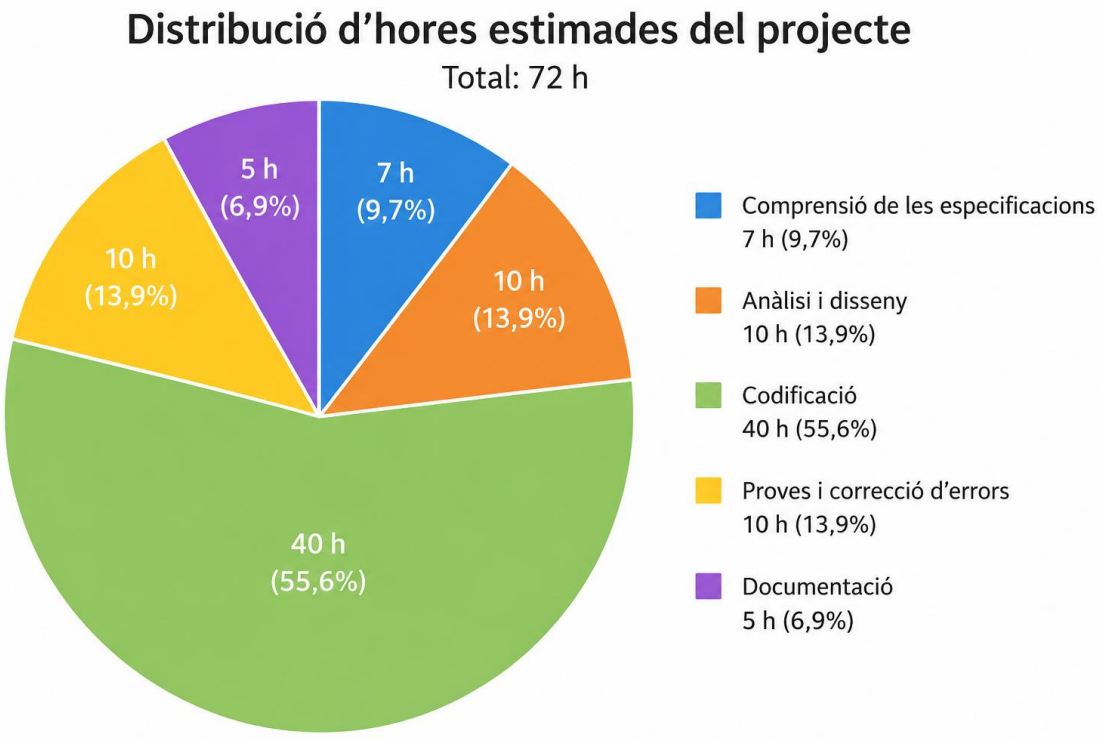



*Figura 13. Distribució de la dedicació del projecte.*

# 10. Conclusions

## 10.1. Conclusions tècniques

A nivell tècnic, aquest projecte ens ha permès aplicar de manera pràctica els conceptes treballats durant l'assignatura, especialment els relacionats amb la programació orientada a objectes, el desenvolupament d'interfícies gràfiques i la gestió de bases de dades. Al llarg del desenvolupament hem treballat amb classes, entitats, controladors, patrons DAO i una arquitectura organitzada per capes, fet que ha facilitat mantenir el codi estructurat, modular i més fàcil de mantenir.

També hem pogut comprendre la importància de la persistència de dades i de la connexió entre l'aplicació Java i la base de dades mitjançant JDBC. La integració entre la interfície gràfica i el sistema d'emmagatzematge ha estat una de les parts més importants del projecte, ja que totes les funcionalitats depenen d'un correcte tractament de les dades.

Durant el desenvolupament també hem après la importància de validar correctament la informació introduïda pels usuaris i d'aplicar restriccions de coherència per evitar errors en el sistema. Aquest aspecte ha estat especialment rellevant en funcionalitats com la reserva de places, l'aparcament de vehicles o la gestió d'usuaris, on era necessari controlar situacions com matrícules duplicades, reserves inconsistents o places ocupades incorrectament.

Un dels aprenentatges més destacats ha estat entendre que molts errors no apareixen durant la compilació, sinó durant les proves reals de funcionament. Diversos problemes només es van detectar simulant casos d'ús amb diferents usuaris i situacions reals, com per exemple conflictes entre reserves, places ocupades "fantasma" o intents d'accedir a dades que no corresponien a l'usuari autenticat. Aquest procés de proves ens ha ajudat a millorar la robustesa i estabilitat de l'aplicació.

Finalment, el projecte també ens ha permès familiaritzar-nos amb eines professionals de desenvolupament, documentació i gestió de bases de dades, així com amb metodologies d'organització del treball per fases i sprints.

## 10.2. Conclusions personals

A nivell personal, el desenvolupament del projecte ha estat una experiència molt útil tant des del punt de vista acadèmic com organitzatiu. El treball en equip ens ha obligat a coordinar-nos, repartir responsabilitats i integrar parts del codi desenvolupades per diferents membres del grup, aspecte que ens ha ajudat a entendre millor com es treballa en projectes de desenvolupament reals.

També hem après la importància de mantenir una bona organització del projecte, documentar les decisions preses i seguir una estructura clara del codi perquè qualsevol membre de l'equip pugui comprendre i modificar fàcilment les diferents parts de l'aplicació. A més, hem comprovat que realitzar proves de manera constant durant el desenvolupament és fonamental per detectar errors abans de les fases finals del projecte.

El projecte ens ha permès millorar les nostres habilitats de resolució de problemes, ja que en moltes ocasions ha estat necessari buscar solucions alternatives, reorganitzar funcionalitats o corregir errors que inicialment

no havíem previst. Aquest procés ens ha ajudat a adquirir una visió més pràctica del desenvolupament de programari i de la importància de la planificació i el manteniment del codi.

# 11. Bibliografia consultada

Les fonts següents s'han consultat com a suport durant el desenvolupament del projecte i la documentació:


ORACLE CORPORATION. *Java Platform, Standard Edition Documentation* [en línea]. [Consulta: maig 2026]. Disponible en: https://docs.oracle.com/en/java/

ORACLE CORPORATION. *Trail: Creating a GUI With Swing* [en línea]. [Consulta: maig 2026]. Disponible en: https://docs.oracle.com/javase/tutorial/uiswing/

ORACLE CORPORATION. *MySQL 8.0 Reference Manual* [en línea]. [Consulta: maig 2026]. Disponible en: https://dev.mysql.com/doc/

THE PHPMYADMIN PROJECT. *phpMyAdmin Documentation* [en línea]. [Consulta: maig 2026]. Disponible en: https://docs.phpmyadmin.net/

JETBRAINS. *IntelliJ IDEA Documentation* [en línea]. [Consulta: maig 2026]. Disponible en: https://www.jetbrains.com/help/idea/

ORACLE CORPORATION. *JDBC Database Access* [en línea]. [Consulta: maig 2026]. Disponible en: https://docs.oracle.com/javase/tutorial/jdbc/

MYSQL. *MySQL Connector/J 8.0 Developer Guide* [en línea]. [Consulta: abril 2026]. Disponible en: https://dev.mysql.com/doc/connector-j/8.0/en/

ATLASSIAN. *Jira Software Documentation* [en línea]. [Consulta: maig 2026]. Disponible en: https://support.atlassian.com/jira-software-cloud/

ORACLE CORPORATION. *Java SE 8 API Specification — java.util* [en línea]. [Consulta: maig 2026]. Disponible en: https://docs.oracle.com/javase/8/docs/api/java/util/package-summary.html

ORACLE CORPORATION. *java.nio.file — Files.readAllBytes* [en línea]. [Consulta: maig 2026]. Disponible en: https://docs.oracle.com/javase/8/docs/api/java/nio/file/Files.html

REFACTORING.GURU. *Singleton Design Pattern* [en línea]. [Consulta: maig 2026]. Disponible en: https://refactoring.guru/design-patterns/singleton

REFACTORING.GURU. *Data Access Object (DAO) Pattern* [en línea]. [Consulta: maig 2026]. Disponible en: https://refactoring.guru/design-patterns/dao

# 12. Annexos

## 12.1. Estructura del projecte

L'estructura del projecte segueix una organització per paquets. A continuació s'ha d'incloure una captura de l'arbre del projecte dins d'IntelliJ IDEA.

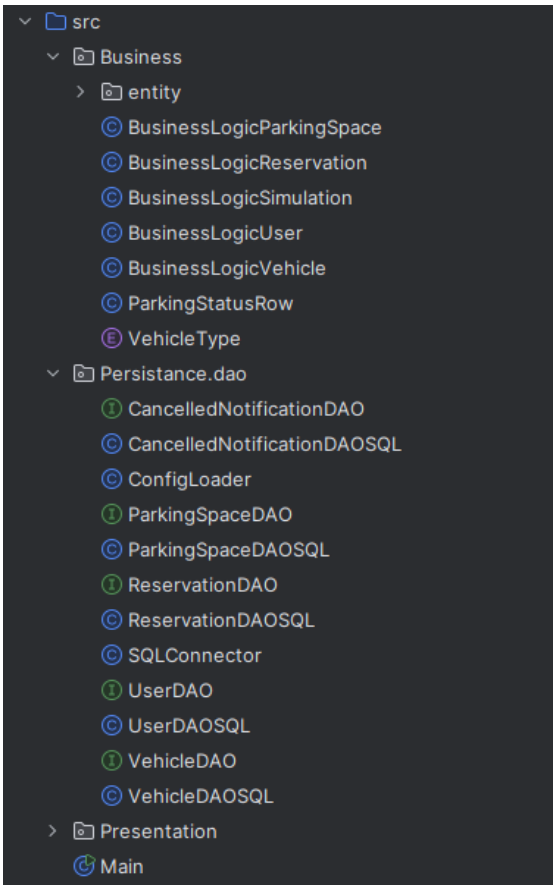



*Figura 14. Estructura de paquets del projecte.*

## 12.2. Instruccions d'execució

Per executar l'aplicació en un entorn nou, cal seguir aquests passos:

1. Crear o importar la base de dades parking_db utilitzant el fitxer SQL exportat.
2. Comprovar que config.json conté les dades correctes de connexió a la base de dades.
3. Obrir el projecte amb IntelliJ IDEA.
4. Comprovar que el connector mysql-connector-java-8.0.23.jar està afegit com a llibreria.
5. Executar la classe Main.
6. Iniciar sessió com a administrador o com a usuari de prova.

## 12.3. Usuaris de prova

| Usuari | Email | Contrasenya | Comentari |
|---|---|---|---|
| admin | admin@parking.com | admin1234 | Usuari administrador definit a config.json. |

| | | | |
|---|---|---|---|
| pau | pau@test.com | Pau1234! | Usuari de prova amb vehicles i reserves. |
| anna | anna@test.com | Anna1234! | Usuari de prova amb vehicles i reserves. |
| marc | marc@test.com | Marc1234! | Usuari de prova. |
| laia | laia@test.com | Laia1234! | Usuari de prova. |
| jordi | jordi@test.com | Jordi1234! | Usuari de prova amb notificació pendent en alguns scripts de prova. |

Nota: les contrasenyes dels usuaris de prova poden variar segons el fitxer SQL final que s'entregui. Cal revisar-les abans del lliurament.